

Concentrate or Spread? Shaping Topological Resampling Priors for Differentiable Triangle-Soup Reconstruction

Viritphon Chongpermwattana-pol Pizzanu Kanongchaiyos
Department of Computer Engineering, Faculty of Engineering,
Chulalongkorn University, Bangkok, Thailand

July 3, 2026

Abstract

Differentiable rasterization reconstructs a 3D triangle soup from images by following photometric gradients while periodically resampling triangles. We first show this pipeline is topologically blind: Chamfer and Hausdorff distances cannot separate reconstructions that differ in genus, connectivity, or enclosed-void structure, while the bottleneck distance between persistence diagrams can; a topological bias supplied at initialization is erased by the first resampling step. Correct topology therefore requires in-loop guidance and a persistence-based metric to detect it. We introduce topology-aware adaptive resampling: a precomputed importance field that localizes the target’s persistent features and biases where new triangles respawn and which are protected from pruning; budget-neutral, it reduces exactly to the baseline when disabled. A single mass-preserving scale interpolates between a *concentrated* prior (importance piled on each feature’s localized death-simplex) and a *spread* prior (the same mass over the feature’s support). We find no dimensional crossover: a spread prior is at least as good as concentrating at every dimension and strictly better for voids and loops—for enclosed voids it cuts bottleneck distance 33–53% below the baseline. The gain is carried by *width* rather than topological placement: a width-matched non-topological control recovers most of it, leaving a small, shape-dependent residual for voids, a small cost on the loop, and a null for components. The reason is that a feature’s death-simplex is a localized witness, not its full support, so the right bias spreads importance over that support. The prescription: *spread*, do not concentrate—the caveat: the benefit comes from width, not topological targeting per se.

1 Introduction

Differentiable rasterization optimizes an explicit 3D representation directly against multi-view images by backpropagating a photometric loss through the rasterizer [18, 20–22]. A *triangle soup*—an unstructured collection of triangles with no shared connectivity—is an attractive such representation [28]: it is cheap to render, places no manifold constraint on the geometry, and supports *adaptive resampling*, a periodic in-loop step that prunes poorly-contributing triangles and respawns new ones to redistribute resolution where the image residual demands it, in the spirit of the adaptive densify-and-prune loops of explicit-primitive and mesh-evolution pipelines [14, 19, 24, 25].

Such pipelines are tuned to minimize geometric error, and geometric error is exactly what they reduce. But geometry and *topology* are different: a reconstruction can be everywhere close to the target surface yet wrong in genus, in the number of connected components, or in whether an enclosed void survives. Our first contribution is to show that the standard pipeline is *topologically blind* in a precise, measurable sense. We construct matched pairs of candidate reconstructions whose Chamfer

distance to the ground truth is *equal by construction* but whose topology differs, and show that Chamfer—and even Hausdorff—cannot separate them, whereas the bottleneck distance between persistence diagrams can (Section 5.1). We further observe that a topological bias injected only at *initialization* is erased: the first resampling step washes it out, so the final topology is independent of the initial bias.

Two consequences follow, and they frame the paper. First, correcting topology requires guidance applied *in the loop*, not at initialization. Second, detecting whether topology is correct requires a metric that sees it—a persistence-based measure rather than a point-to-point distance.

We introduce *topology-aware adaptive resampling* (Section 3): a precomputed importance field that localizes the target’s persistent features and biases (i) where new triangles respawn and (ii) which triangles are protected from pruning. The method is budget-neutral (the triangle count is held fixed) and reduces *exactly* to the baseline when its single strength knob is set to zero. Within this one mechanism we vary a *concentrate*↔*spread* scale on the importance field, which lets us ask not merely whether topological guidance helps, but *what spatial form* the guidance should take.

Our central finding concerns the *spatial form* of that guidance. A topological prior helps voids (H2), but chiefly through *width*: a spread prior lowers bottleneck-to-target by 33–53% across spheres, cubes, and capped cylinders, yet a width-matched non-topological control recovers most of it—so the void gain is mainly a spread effect, with only a small, shape-dependent topological residual (Section 5.2). But the natural hypothesis—that high-dimensional features want a *concentrating* prior and low-dimensional ones a *spreading* one, a dimensional crossover—turns out to be *false*. Comparing the two priors directly (Section 5.3), a spread prior is at least as good as concentrating at every dimension and strictly better for voids and loops; concentrating on a one-dimensional loop actively backfires, over-tessellating it into phantom handles. Across dimensions the benefit tracks *width*, not topological placement—a width-matched non-topological prior does at least as well (better, on the loop)—so the operative takeaway is to *spread* resampling resolution, not to target topology per se. The unifying reason (Section 6) is that a feature’s death-simplex is only a localized witness of its support, so spreading importance over that support is the right bias—while the *harm* from concentrating is what depends on dimension. This turns “does topology help?” into a prescription for how to shape the prior.

2 Related work

Topology in 3D reconstruction. Topological correctness in reconstruction has mainly been pursued along three lines. Interactive, constraint-driven reconstruction resolves topological ambiguity during fitting, deferring to a user where the data cannot decide genus or connectivity [27]. Mesh repair removes topological defects—holes, spurious handles, disconnected shells—from an already-built mesh [2, 17]. Closest to our setting, persistence-based priors have been imposed while fitting an implicit representation to a point cloud, promoting a prescribed persistence diagram during optimization [4]. All of these presuppose either explicit connectivity (something to repair or constrain) or a global surface fit that can absorb a topological penalty. A triangle soup under differentiable rasterization offers neither: there is no connectivity to repair, and we show (Section 5.1) that a topological bias present only at initialization is erased by the first resampling step—so guidance has to act inside the resampling loop itself.

Persistent-homology-based losses and priors. Persistent homology can serve as an optimizable objective: differentiable persistence layers [5], topological loss terms for image segmentation [7, 16], persistence-driven shape matching [26], and a general convergence analysis for

persistence-based objectives [6]. In all of these the topological signal reaches the parameters as a *gradient*, at the price of a persistence computation in every step. Our importance field is deliberately weaker: the target’s diagram is computed *once*, offline, and only steers where the resampler spends its fixed triangle budget; nothing is differentiated through persistent homology. This makes the prior essentially free at training time and isolates the question studied here—*where* topological guidance should place resolution—from how a differentiable topological signal would be constructed (the natural next phase; Section 7).

Differentiable rendering / rasterization. Differentiable rasterizers obtain image-space gradients for explicit geometry, from approximate derivatives [18, 22] to smoothed coverage [21] and high-performance modular primitives [20]. On top of them, explicit-primitive pipelines evolve their primitive set *in the loop*: 3D Gaussian splatting densifies and prunes by view-space gradient magnitude [19], triangle splatting carries the same adaptive-density recipe to triangle primitives [14], and mesh-evolution methods remesh inside the inverse-rendering loop [24, 25]. In every case the densification heuristic is photometric or geometric—screen-space error, gradient magnitude, edge length—and none targets topology. The triangle-soup pipeline we build on [28] inherits exactly this heuristic, which is what makes it topologically blind (Section 5.1).

Remeshing and adaptive sampling. Allocating resolution adaptively is classical: progressive meshes and quadric-error simplification concentrate triangles where a geometric error metric demands [13, 15], and remeshing methods adapt triangle density to curvature or approximation error, interactively or in real time [1, 3, 9]. Our resampling policy keeps this machinery and swaps the *signal*: importance derives from the target’s persistent features rather than from curvature or error. The non-topological controls B3/B5 (Section 4) are precisely the generic-densification analogue of such heuristics at matched kernel width—which is what lets the experiments separate “more resolution, well spread” from “resolution placed by topology.”

3 Topology-aware adaptive resampling

The method has two offline-then-online parts: a *precomputed importance field* distilled once from the target, and a *resampling policy* that consumes the field inside the training loop. Neither touches the renderer, and both are budget-neutral.

3.1 The importance field

Let P be a seeded surface sample of the target. We compute its persistence diagram [12] with a GUDHI [23] alpha complex [11] and keep the *significant* features—intervals whose lifetime exceeds a self-calibrating multiple of the median nearest-neighbour spacing. Each significant feature f of homological dimension d_f is *localized* in space using the persistence pairs together with the alpha-complex point coordinates: an H1 loop is located by the edge/triangle that births/fills it, an H2 void by the tetrahedron that fills it (which sits on the bounding shell), and an H0 component by the edge that merges it. This yields, per feature, a set of 3D kernel centers $\{c\}_f$ on the target surface.

Each feature contributes an isotropic Gaussian splat, and the field is evaluated on a dense surface sample Q (an “option-B surface splat”) so that importance is tied to the manifold the soup

must reproduce:

$$r(q) = \sum_f w_f \sum_{c \in \{c\}_f} \exp\left(-\frac{\|q-c\|^2}{2\sigma_f^2}\right), \quad q \in Q, \quad (1)$$

with per-feature weight w_f equal to the (normalized) persistence and radius σ_f set to the feature’s own death scale—its alpha circumradius $\sqrt{\text{death}_f}$, clamped to a spacing floor and a bounding-box-fraction ceiling. The per-dimension fields are normalized to $[0, 1]$ and a query point takes the value of its nearest sample in Q .

Concentrate $\leftrightarrow$ spread scale. We add a single scale $s \geq 1$ that widens every kernel while preserving the injected surface mass: $\sigma_f \rightarrow s\sigma_f$ and $w_f \rightarrow w_f/s^2$. Because a 2D Gaussian’s surface integral scales as $w\sigma^2$, the total importance mass $\sum_f w_f\sigma_f^2$ is invariant in s ; s changes only the spatial *spread* of the prior, not how much of it there is. By construction $s=1$ is the *concentrated* field and is numerically identical to the field used without the knob.

3.2 The resampling policy

The baseline resampler [28] periodically (i) *prunes* the least screen-visible triangles and (ii) *respawns* triangles by subdividing large screen-space edges. Our policy biases both with the field via one strength knob λ_{topo} , and keeps the triangle count N fixed:

- **Prune protection (keep-map).** Faces are ranked by visibility + $\lambda_{\text{topo}} \kappa \text{imp}$, where imp is the field value at the face centroid and κ a visibility-scale factor; the same number of faces is removed as in the baseline, so important faces are protected without changing the budget.
- **Budget-neutral densification.** After the unchanged baseline split, we add up to $\text{cap} = \lambda_{\text{topo}} \cdot c \cdot N$ extra subdivisions of the highest-importance visible faces, then remove exactly cap faces with the protective keep-map. Net face count is unchanged; the effect is a pure trade of low-importance triangles for high-importance ones.

When $\lambda_{\text{topo}} = 0$ both levers are bypassed and the policy reduces *exactly* to the baseline resampler (bit-identical resampling decisions); this is condition B0 below. We use our own implementation of this pipeline [28]; since the $\lambda_{\text{topo}}=0$ policy reproduces its baseline resampler bit-identically, all reported numbers are with respect to this implementation. The policy is field-agnostic: swapping which field it reads (concentrated topological, spread topological, or a non-topological control) gives the different experimental conditions of Section 4 without any other change.

4 Experimental setup

Conditions. All conditions are identical in seed, target scene, triangle budget N , optimizer, step count, and schedule; they differ *only* in the resampling importance (Section 3.2).

- **B0** — baseline resampler ($\lambda_{\text{topo}} = 0$); the control.
- **B1** — topological field applied *once at initialization* only; tests whether init-time bias survives (it does not).
- **B2** — topological field applied in-loop, *concentrated* ($s=1$); the proposed method.

- **B3** — a *non-topological* importance field (random blobs at the default *narrow* width) applied in-loop; with B2 it tests whether narrow topological placement beats narrow non-topological densification.
- **B4** — topological field applied in-loop but *spread* ($s > 1$); identical to B2 except for the $\text{concentrate} \leftrightarrow \text{spread}$ scale. *Results in Sections 5.2 and 5.3.*
- **B5** — a *non-topological* field *width-matched* to B4 (random blobs widened to B4’s kernel σ , mass-preserved); with B3 and B4 it separates topological *placement* from kernel *width* (H2 shapes; Section 5.2).

Synthetic targets with known topology. Targets are analytic shapes rendered into multi-view scenes so the ground-truth surface (and hence its Betti numbers) is exact. Each shape’s Betti numbers are recovered by the alpha-complex pipeline before any reconstruction experiment.

target	(b_0, b_1, b_2)	decisive dim	role
sphere	(1, 0, 1)	H2	enclosed void
cube	(1, 0, 1)	H2	void, flat faces / sharp edges
cylinder	(1, 0, 1)	H2	void, capped
torus	(1, 2, 1)	H1	loops / handle
two_spheres	(2, 0, 2)	H0	two components
three_spheres	(3, 0, 3)	H0	three components
double_torus	(1, 4, 1)	H1	genus-2, multi-loop

Table 1: Synthetic targets and their (verified) Betti numbers.

Metric. The primary metric is the *bottleneck distance* [8, 10] between the reconstruction’s persistence diagram and the target’s, in the shape’s decisive homology dimension, reported as the tail mean over the last 30% of training steps (lower is better). Geometry is reported in parallel as Chamfer / Hausdorff-95 (% of bbox diagonal) so that any topological gain can be checked to occur at *comparable* geometry rather than from a geometric change.

Determinism and seeds. Resampling decisions (the face structure F) are deterministic given the seed, but the CUDA rasterizer’s gradient atomics make vertex positions V *not* bit-reproducible run-to-run: two same-seed baseline runs can drift by a small number of triangles. We therefore (i) use *paired* seeds across conditions and (ii) average each comparison over multiple seeds—5 for the H2 study and the torus, 3 for the remaining crossover shapes; the topology metric samples the soup $\propto$ area-weighted and is robust to a few drifting triangles.

Budget. All comparisons are budget-neutral: the triangle count N is held fixed across conditions and across the run (the densify step trades faces 1-for-1). Training runs 2500 steps, at $N=1200$ for the H2 study and at each shape’s headroom budget for the crossover (Table 4).

5 Results

5.1 Geometric metrics are topologically blind

We build three controlled pairs (A = topologically correct, B = topologically wrong). In each pair A’s geometry is bisection-matched so its Chamfer distance to the ground truth *equals* B’s; topology is the only thing that differs. Diagrams are computed on 40,000 seeded surface samples per shape.

case	candidate	Chamfer %	Hausd95 %	bott→GT (disc)	disc dim
i_genus	A: blister, genus 0 (correct)	0.916	0.481	0.0000	H1
	B: thin handle, genus 1 (wrong)	0.916	0.456	0.0302	H1
ii_components	A: two components (correct)	0.628	0.536	0.0014	H0
	B: bridged, one comp. (wrong)	0.628	0.357	0.0574	H0
iii_void	A: blister, void intact (correct)	0.977	1.887	0.0040	H2
	B: punctured, void gone (wrong)	0.977	4.375	0.1385	H2

Table 2: Geometric metrics cannot separate topologically distinct reconstructions, while the discriminating-dimension bottleneck does. Chamfer is equal within each pair by construction; in cases i and ii Hausdorff-95 even *prefers* the wrong candidate. bottleneck→GT is $\approx 40\times$ (H0) and $\approx 35\times$ (H2) larger for the wrong candidate.

In all three cases Chamfer ranks the wrong candidate equal-or-better while the bottleneck distance separates them cleanly (Figure 1). The blindness holds even at the level of geometry’s most outlier-sensitive metric: Hausdorff-95 is *lower* (better) for the wrong candidate in cases i and ii.

A topological bias supplied only at initialization does not survive training: the first resampling step removes it, so the final topology is independent of the initial bias. The persistence-based stability series makes the gap visible—Chamfer drifts smoothly while the bottleneck jumps when a spurious feature becomes significant (Figure 2).

5.2 Voids (H2): mostly width, with a shape-dependent topological residual

For voids, spreading the resampling prior helps substantially—a controlled 2×2 over *kernel width* (narrow $s=1$ vs. wide $s=3$) and *placement* (topological vs. non-topological; Table 3) shows the gain is chiefly a *width* effect with a smaller, shape-dependent topological component. *Width*: at matched placement, widening cuts error by 15–33% (topological B4-vs-B2 by 18–33%, non-topological B5-vs-B3 by 15–33%), so both *wide* priors land far below both *narrow* ones. (Not every wide-vs-narrow *cross*-pair clears 15%—e.g. B5-vs-B2 is $\approx 9\%$ for the cylinder—so the range is stated for matched-placement widening.) *Placement*: the topological-placement benefit is real but shape-dependent, clearest for the cylinder, where topology helps at *both* widths (narrow B2-vs-B3 by 7.8%; wide B4-vs-B5 by $\approx 4.6\sigma$); for the sphere it is weak (narrow: a tie; wide: $\approx 2.2\sigma$), and for the cube absent (a tie at both widths). B4 is the best single condition at better-or-equal geometry, but a width-matched non-topological control recovers most of its advantage, so the void gain is chiefly width with a real but shape-dependent topological residual. Init-time bias washes out (B1 $\approx$ B0).

Whether this “spread beats concentrate” ordering is special to voids or holds across feature dimensions is the subject of Section 5.3.

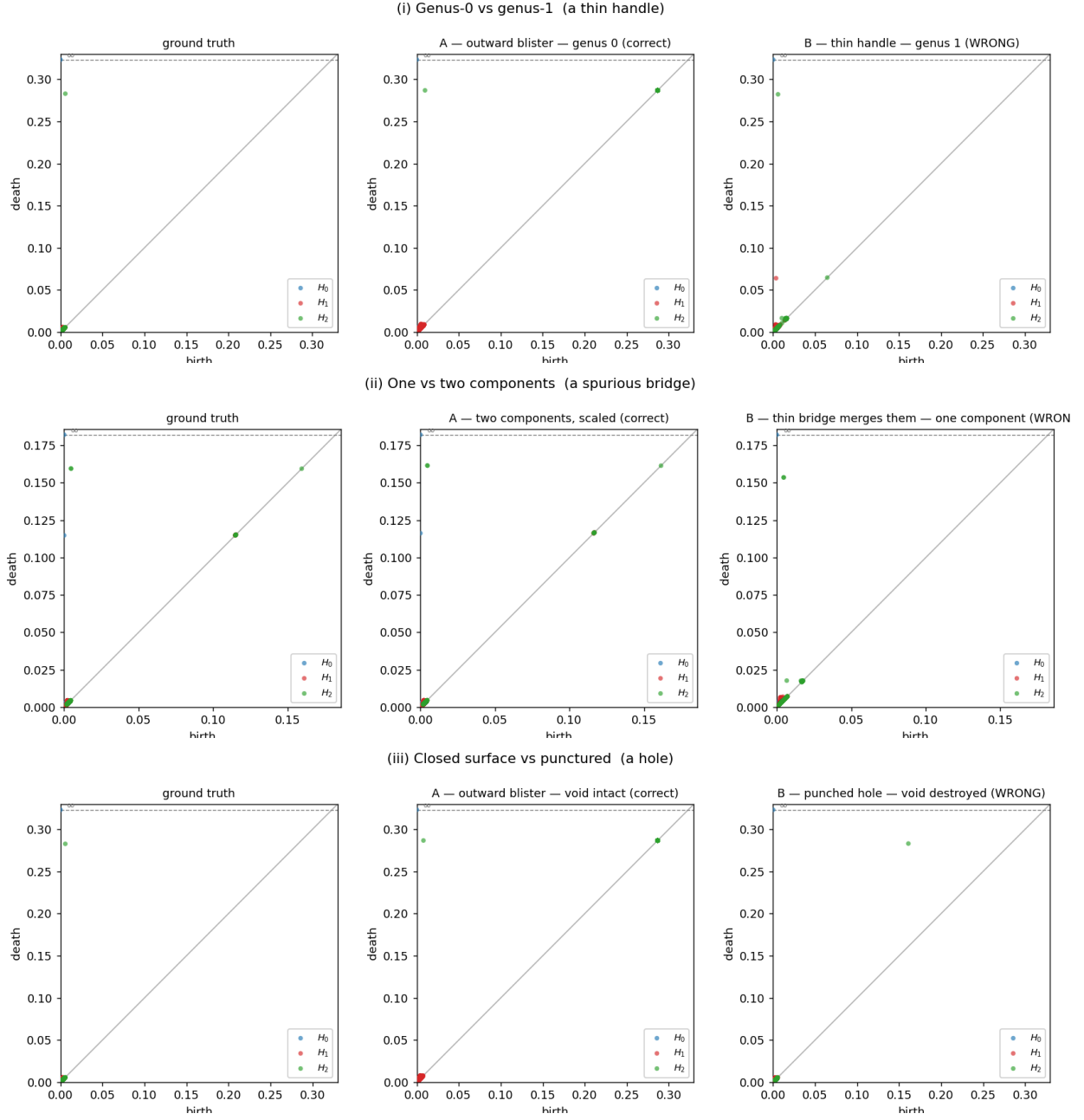


Figure 1: Persistence diagrams (GT / A / B) for the three blindness cases, top to bottom: (i) genus, (ii) components, (iii) void.

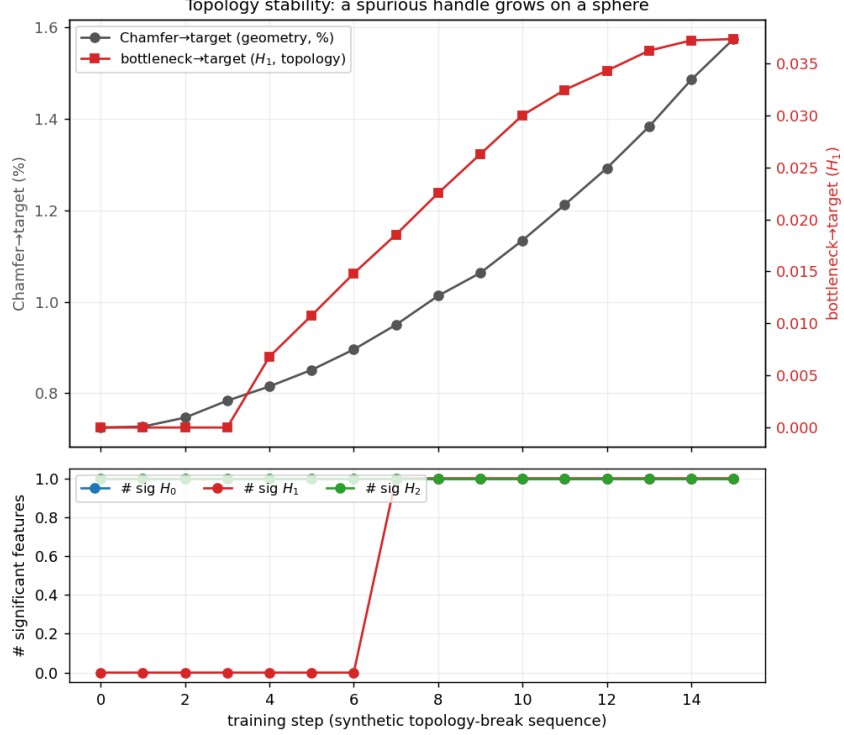


Figure 2: Topology stability series: Chamfer (smooth) vs bottleneck- H_1 (jumps when a spurious handle becomes significant).

5.3 Does the optimal prior flip with dimension?

Section 5.2 shows spread beats concentrate for voids. Does a low-dimensional feature instead prefer concentration—a *dimensional crossover*? We compare the concentrated (B2) and spread (B4) priors—identical but for the width scale—in each shape’s decisive dimension, at a per-class *headroom* budget, and track the signed difference $\Delta = \text{bottleneck}(\text{B4}) - \text{bottleneck}(\text{B2})$ (Table 4); a crossover would show Δ change sign with dimension. We add a genus-2 `double_torus` ($b_1=4$) and `three_spheres` ($b_0=3$).

No crossover; spread wins or ties everywhere. $\Delta = \text{B4} - \text{B2}$ (spread minus concentrate, both *topological*) is clearly negative for voids and the torus (spread better) and only marginally negative for the two H_0 shapes—within noise of the non-topological control, i.e. a null. Nowhere does concentrating win, so the optimal spatial prior does not flip with dimension. Because this Δ holds placement fixed (both priors topological), it isolates the effect of *width*; whether the loop spread-win is *topological* needs the non-topological arm, which we add for the torus next (Table 5).

On the loop, width drives the gain; topological placement does not. A $\text{width} \times \text{placement}$ 2×2 on the torus (Table 5), mirroring the void analysis, decomposes the spread-win. *Width* helps massively at either placement: widening drops the bottleneck 26% for the topological pair (B2→B4) and 33% for the non-topological pair (B3→B5), putting both *wide* cells far below both *narrow* ones. *Placement*, by contrast, gives no benefit on the loop—at both widths the *non-topological* prior is better: narrow B3 $0.0397 < \text{B2 } 0.0425$ (7.6σ) and wide B5 $0.0267 < \text{B4 } 0.0316$ (3.25σ), so the best

target (H2)	narrow ($s=1$)		wide ($s=3$)		B0 base	$\Delta(B4-B5)$
	B2 topo	B3 non-topo	B4 topo	B5 non-topo		
sphere	0.0438	0.0440	0.0357	0.0374	0.0535	-0.0017 (2.2σ)
cube	0.0409	0.0403	0.0273	0.0270	0.0579	$+0.0003$ (tie)
cylinder	0.0440	0.0477	0.0360	0.0402	0.0545	-0.0042 (4.6σ)

Table 3: Voids, one paired-seed run ($N=1200$, 5 seeds): bottleneck-to-target (decisive dim, tail mean; lower better) in a width $\times$ placement 2×2 . At matched placement, *widening* cuts error by 15–33% (B4-vs-B2, B5-vs-B3); *placement* is a smaller, shape-dependent residual—the topological column beats the non-topological one clearly for the cylinder (both widths), weakly for the sphere (wide only), and not for the cube. $B0$ is the baseline; $B1\approx B0$ (init washout, omitted).

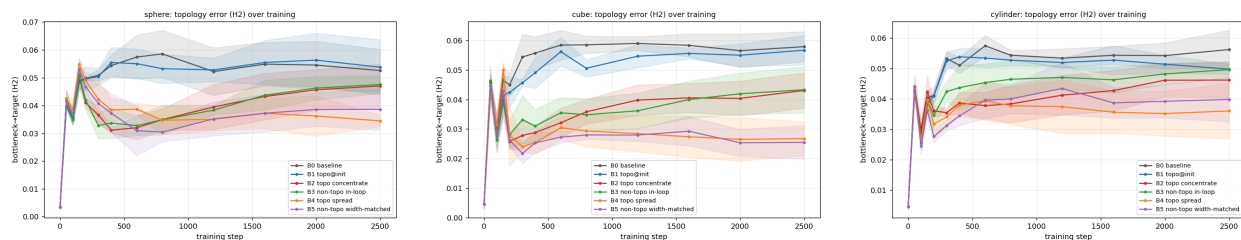


Figure 3: Bottleneck-to-target over training for the three voids (conditions B0–B5): the two *wide* priors (B4 topological, B5 non-topological) track well below the two *narrow* priors (B2, B3), and B4/B5 nearly coincide—width, not topology, is the driver. Geometry parity: Figure 4.

torus condition is a *wide non-topological* prior. The only distinctly topological effect is a harmful one: at this budget only the narrow-topological prior (B2) inflates significant-H1 into phantom handles (4.4 vs. a true 2; B3, B4, and B5 all hold 2.0), the concentrate backfire of Section 6. So on the torus, as on the voids, *width* drives the gain—but here topological placement is a small cost, not a small residual.

Components are a null. For `two_spheres` and `three_spheres` neither topological prior beats the non-topological control: component correctness is governed by the *absence* of geometry in the inter-component gap, which a placement prior does not control (Section 6).

Multi-loop genus-2: inconclusive. On `double_torus` the decisive-dimension bottleneck is identical across all conditions (pinned at 0.0264) and the soup recovers only 2 of the 4 loops at every budget tried, so this stress test does not discriminate; we report it as inconclusive rather than support, and flag a higher-resolution re-run as future work.

6 Why spreading wins, and why concentration’s harm depends on dimension

What the importance scale actually varies. A feature’s death simplex marks *where* its topology is fragile, but the feature’s support is larger—the bounding shell of a void, the whole tube of a handle. For the compact targets used here the feature’s death scale is of order the shape size,

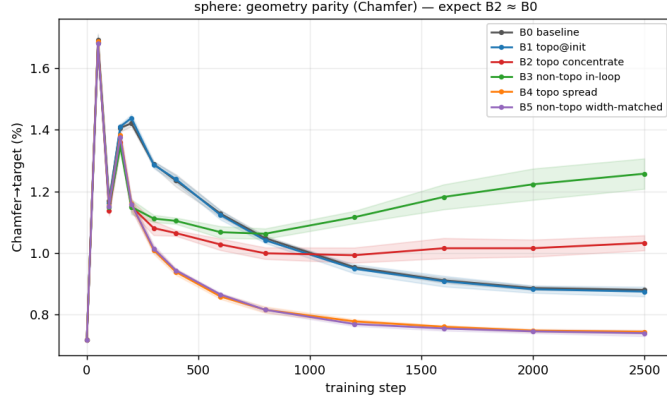


Figure 4: Sphere geometry (Chamfer): B4 (spread) sits at or below baseline, so the topological gain is not bought with geometry.

target	dim	N	B2 concentrate	B4 spread	$\Delta = B4 - B2$
sphere	H2	1200	0.0438	0.0357	-0.0081
cube	H2	1200	0.0409	0.0273	-0.0136
cylinder	H2	1200	0.0440	0.0360	-0.0080
torus	H1	700	0.0425	0.0316	-0.0109
two_spheres	H0	700	0.0059	0.0057	-0.0002
three_spheres	H0	700	0.0062	0.0053	-0.0009
double_torus	H1	2000	inconclusive (bottleneck pinned; see text)		

Table 4: Concentrate vs. spread across feature dimensions: $\Delta \leq 0$ *everywhere*, so there is no sign flip and no crossover—spread is at least as good as concentrating at every dimension, and strictly better for voids and the torus. The H2 rows are the paired-seed run of Table 3 (h2_unified, 5 seeds); the low-dimensional rows are the crossover run (torus 5 seeds, others 3). Each row is internally consistent (one run per shape).

so even the concentrated field ($s=1$) already overlaps most of the surface. The scale s therefore varies the *peakedness* of the field—and hence the differential protection the keep-map applies to feature geometry—far more than its literal spatial coverage. “Spreading” here means “protect the feature’s whole support evenly,” not “reach more of the surface.”

Voids: spreading (width) helps; topological placement is a small residual. An enclosed void is destroyed by a hole *anywhere* on the 2-manifold shell that bounds it, so the void’s support—the geometry whose failure collapses it—is the *entire* shell, which for a compact shape is essentially the whole surface. As the peakedness caveat above notes, even the concentrated field ($s=1$) already overlaps most of that surface, so *any* wide field—topological or not—approximately spreads over the void’s support. The single principle “spread over the support” therefore predicts the void effect should look width-driven, with topological placement only a residual—and it is: a width-matched non-topological control (B5) recovers most of the spread prior’s gain (Section 5.2). That residual—placing the wide prior at the void locus (B4) rather than at random (B5)—should track how *non-uniform* the bounding shell is, and it does: largest for the cylinder (flat caps and a curved side; $\approx 4.6\sigma$), smaller for the near-uniform sphere ($\approx 2.2\sigma$), and absent for the cube (a tie).

	narrow ($s=1$)		wide ($s=3$)		
torus (H1), $N=700$	B2 topo	B3 non-topo	B4 topo	B5 non-topo	B0
bottleneck (tail mean)	0.0425	0.0397	0.0316	0.0267	0.0409
significant-H1 (true 2)	4.4	2.0	2.0	2.0	2.0

Table 5: Torus width $\times$ placement 2×2 (5 paired seeds, $N=700$), mirroring Table 3 for a loop. *Width* helps at either placement (both wide cells far below both narrow); at matched *placement* the *non-topological* prior is better at both widths (narrow 7.6σ , wide 3.25σ), so the loop spread-win is a width effect. Topological placement’s only distinct effect is harmful: the narrow-topological B2 alone inflates significant-H1 into phantom handles.

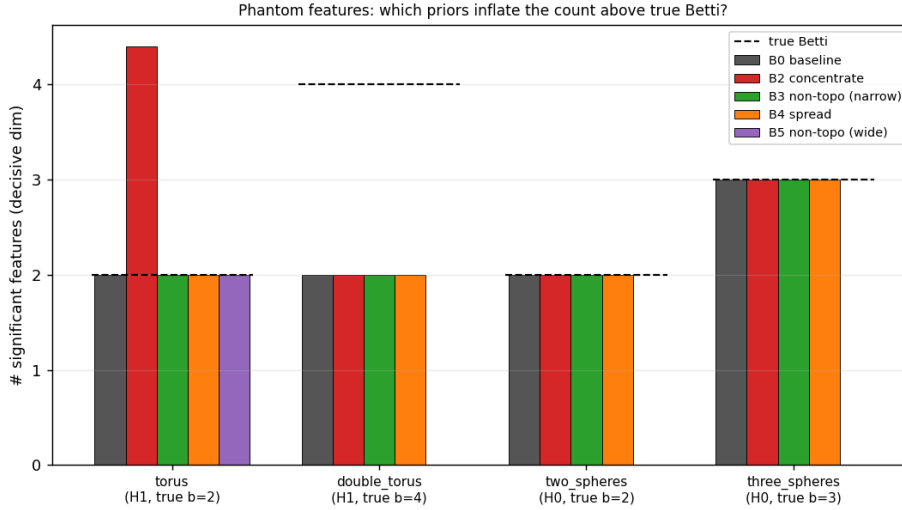


Figure 5: Significant-feature count vs. true Betti (low-dimensional shapes): concentrate (B2) inflates the torus H1 count (phantom handles), while spread (B4) *and* both non-topological priors (narrow B3, wide B5; Table 5) hold the true value—so the inflation is specific to the narrow-topological prior; the H0 shapes are unaffected. (B5 was run on the torus only.)

The harm of concentrating, and why it scales with dimension. What *does* depend on dimension is the damage a concentrating prior does, and it is governed by how readily over-tessellating a *single locus fabricates a spurious feature of the same dimension*:

- **Loops (1-D): catastrophic, and here placement matters.** A loop’s support is a thin tube, far smaller than the surface, so a *narrow* prior cannot spread along it and over-tessellates it into phantom handles—the significant-H1 count rises above the true value and the bottleneck worsens *below baseline* (Section 5.3). At the headroom budget this backfire is *narrow-topological*-specific: only the narrow-topological prior inflates the count ($N=700$: B2 4.4 vs. narrow-random B3 2.0; Table 5). Only under severe budget starvation does narrowness alone begin to over-tessellate a random prior too ($N=400$, 3 seeds: B2 ≈ 7.7 , B3 ≈ 5.7 , true value 2; Figure 6). Either way the cure is *width*: a wide prior, topological or not, recovers the true count.
- **Voids (2-D): mild.** Over-tessellating one shell patch cannot *create* a second enclosed void, so concentrating fabricates no spurious H2; it merely under-protects the rest of the shell—an

incomplete-protection shortfall behind spreading, never worse than baseline.

- **Components (0-D): none.** A component boundary is defined by the *absence* of geometry between shells; no placement of triangles on the shells adds or removes a component, so a placement prior has no purchase and the result is a null—neither topological prior beats the non-topological control (Section 5.2).

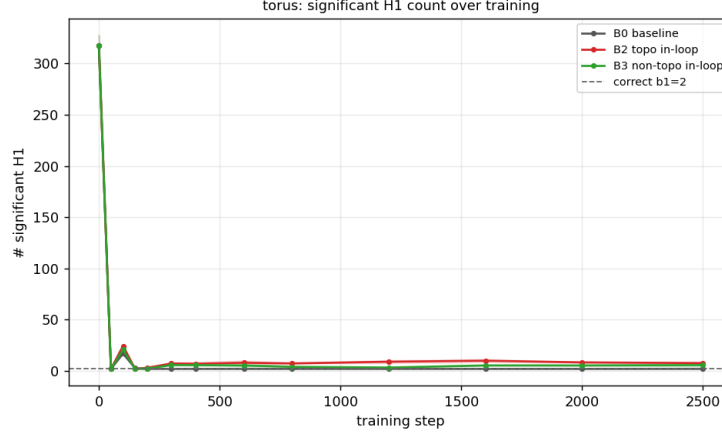


Figure 6: Independent budget sweep, torus at the tight budget $N=400$: *both* narrow priors inflate the significant-H1 count above the true value (phantom handles)—the narrow-topological B2 more than the narrow-random B3—while the baseline recovers the correct count. The over-tessellation is a narrowness effect, worsened by topological placement.

Interpretation: one principle, two observable regimes. A single principle organizes all three cases—spread importance over the feature’s *support*, not its localized death simplex—and, together with the peakedness caveat, it predicts *why* voids look width-driven while loops look narrowness-driven: the difference is how much of the surface the support occupies. A void’s support is the whole bounding shell ($\approx$ the entire compact surface), so any wide field already spreads over it and topological placement is only a residual (Section 5.2). A loop’s support is a thin tube, far smaller than the surface: a wide field does not specifically spread along that tube, and a *narrow* field over-tessellates it into phantom handles—so here narrowness is harmful and the wide (spread) prior recovers the correct topology (Section 5.3). The optimal prior is thus spread at every dimension (no crossover); only the observable signature—and the harm from being narrow—depends on the support’s extent. The genus-2 multi-loop case did not resolve at feasible budgets and is reported as inconclusive.

7 Limitations and future work

Synthetic targets. All experiments use analytic shapes rendered into multi-view scenes, chosen so the ground-truth surface and its Betti numbers are exact and so geometry can be held fixed while topology varies—the control that makes the blindness result and the topological-vs-non-topological comparison clean. The cost is external validity: we do not yet demonstrate the method on captured or noisy real scenes, where the target persistence diagram itself must be estimated.

Non-determinism of the optimizer. The CUDA rasterizer’s gradient atomics make vertex positions non-reproducible run-to-run; resampling structure (F) is deterministic but vertex positions (V) are not, so single-run differences can include a few drifting triangles. We mitigate with paired seeds and seed-averaging and a soup-sampled topology metric, but absolute numbers carry this run-to-run variance.

What the spread scale varies (limited dynamic range). For the compact targets used here, topological death scales are of order the shape size, so even the concentrated field ($s=1$) already overlaps most of the surface. The scale s therefore controls the *peakedness* of the importance field—and thus the differential protection applied by the keep-map—rather than its literal spatial coverage; our claims about “spreading” should be read in that sense. Characterizing the effect on targets whose features are small relative to the shape, where coverage and peakedness decouple, is left to future work.

Width vs. topological placement. The void gain is primarily a kernel-width effect: a width-matched non-topological control recovers most of it, and the residual attributable to topological placement is small and shape-dependent—clear for the cylinder, modest for the sphere, absent for the cube (Section 5.2). Establishing that residual robustly, and whether it grows for targets with finer or off-centre voids, would need more shapes and seeds.

Precomputed, non-differentiable prior. The importance field is distilled once from the target and is *not* differentiated through; the method biases sampling, it does not flow a topological gradient. Making persistent homology differentiable so the topological signal can backpropagate into vertex positions—rather than only steer resampling—is the natural next phase, building on existing differentiable-persistence machinery [5, 6, 26].

Resolving multi-loop / genus-2 targets. Our genus-2 `double_torus` stress test was inconclusive: at every feasible budget the soup recovered only 2 of its 4 loops and the decisive-dimension bottleneck did not discriminate between conditions (Section 5.3). Strengthening the spread-vs-concentrate finding on a richer multi-loop target needs a higher-resolution or longer-schedule re-run, which we leave to future work.

References

- [1] Pierre Alliez, David Cohen-Steiner, Olivier Devillers, Bruno Lévy, and Mathieu Desbrun. Anisotropic polygonal remeshing. *ACM Transactions on Graphics*, 22(3):485–493, 2003.
- [2] Marco Attene. A lightweight approach to repairing digitized polygon meshes. *The Visual Computer*, 26(11):1393–1406, 2010.
- [3] Mario Botsch and Leif Kobbelt. A remeshing approach to multiresolution modeling. In *Eurographics/ACM SIGGRAPH Symposium on Geometry Processing (SGP)*, 2004.
- [4] Rickard Brüel-Gabrielsson, Vignesh Ganapathi-Subramanian, Primoz Skraba, and Leonidas J. Guibas. Topology-aware surface reconstruction for point clouds. *Computer Graphics Forum*, 39(5):197–207, 2020.

- [5] Rickard Brüel Gabrielsson, Bradley J. Nelson, Anjan Dwaraknath, and Primoz Skraba. A topology layer for machine learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 108 of *Proceedings of Machine Learning Research*, pages 1553–1563, 2020.
- [6] Mathieu Carrière, Frédéric Chazal, Marc Glisse, Yuichi Ike, Hariprasad Kannan, and Yuhei Umeda. Optimizing persistent homology based functions. In *International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, 2021.
- [7] James R. Clough, Nicholas Byrne, Ilkay Oksuz, Veronika A. Zimmer, Julia A. Schnabel, and Andrew P. King. A topological loss function for deep-learning based image segmentation using persistent homology. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(12):8766–8778, 2022.
- [8] David Cohen-Steiner, Herbert Edelsbrunner, and John Harer. Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1):103–120, 2007.
- [9] Marion Dunyach, David Vanderhaeghe, Loïc Barthe, and Mario Botsch. Adaptive remeshing for real-time mesh deformation. In *Eurographics 2013 — Short Papers*, 2013.
- [10] Herbert Edelsbrunner and John L. Harer. *Computational Topology: An Introduction*. American Mathematical Society, 2010.
- [11] Herbert Edelsbrunner and Ernst P. Mücke. Three-dimensional alpha shapes. *ACM Transactions on Graphics*, 13(1):43–72, 1994.
- [12] Herbert Edelsbrunner, David Letscher, and Afra Zomorodian. Topological persistence and simplification. *Discrete & Computational Geometry*, 28(4):511–533, 2002.
- [13] Michael Garland and Paul S. Heckbert. Surface simplification using quadric error metrics. In *Proceedings of SIGGRAPH 97*, pages 209–216, 1997.
- [14] Jan Held, Renaud Vandeghen, Adrien Delière, Abdullah Hamdi, Silvio Giancola, Anthony Cioppa, Andrea Vedaldi, Bernard Ghanem, Andrea Tagliasacchi, and Marc Van Droogenbroeck. Triangle splatting for real-time radiance field rendering. arXiv:2505.19175, 2025.
- [15] Hugues Hoppe. Progressive meshes. In *Proceedings of SIGGRAPH 96*, pages 99–108, 1996.
- [16] Xiaoling Hu, Fuxin Li, Dimitris Samaras, and Chao Chen. Topology-preserving deep image segmentation. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
- [17] Tao Ju. Robust repair of polygonal models. *ACM Transactions on Graphics*, 23(3):888–895, 2004.
- [18] Hiroharu Kato, Yoshitaka Ushiku, and Tatsuya Harada. Neural 3D mesh renderer. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3907–3916, 2018.
- [19] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):139:1–139:14, 2023.

- [20] Samuli Laine, Janne Hellsten, Tero Karras, Yeongho Seol, Jaakko Lehtinen, and Timo Aila. Modular primitives for high-performance differentiable rendering. *ACM Transactions on Graphics*, 39(6):194:1–194:14, 2020.
- [21] Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. Soft rasterizer: A differentiable renderer for image-based 3D reasoning. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 7708–7717, 2019.
- [22] Matthew M. Loper and Michael J. Black. OpenDR: An approximate differentiable renderer. In *European Conference on Computer Vision (ECCV)*, pages 154–169, 2014.
- [23] Clément Maria, Jean-Daniel Boissonnat, Marc Glisse, and Mariette Yvinec. The Gudhi library: Simplicial complexes and persistent homology. In *International Congress on Mathematical Software (ICMS)*, volume 8592 of *Lecture Notes in Computer Science*, pages 167–174, 2014.
- [24] Baptiste Nicolet, Alec Jacobson, and Wenzel Jakob. Large steps in inverse rendering of geometry. *ACM Transactions on Graphics*, 40(6), 2021.
- [25] Werner Palfinger. Continuous remeshing for inverse rendering. *Computer Animation and Virtual Worlds*, 33(5):e2101, 2022.
- [26] Adrien Poulenard, Primož Skraba, and Maks Ovsjanikov. Topological function optimization for continuous shape matching. *Computer Graphics Forum*, 37(5):13–25, 2018.
- [27] Andrei Sharf, Thomas Lewiner, Gil Shklarski, Sivan Toledo, and Daniel Cohen-Or. Interactive topology-aware surface reconstruction. *ACM Transactions on Graphics*, 26(3):43:1–43:10, 2007.
- [28] Kenji Tojo, Bernd Bickel, and Nobuyuki Umetani. DiffSoup: Direct differentiable rasterization of triangle soup for extreme radiance field simplification. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026. arXiv:2603.27151.